COMPUTER ENGINEERING

QUEZON CITY

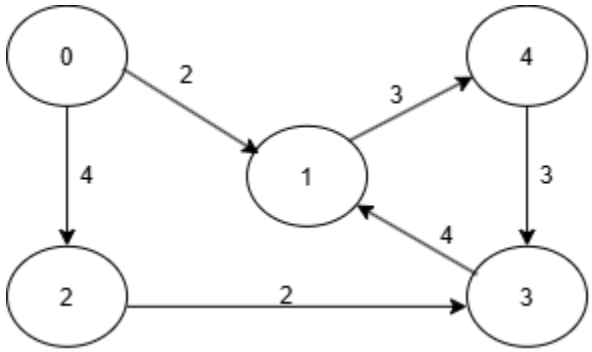
COURSE CODE/TITLE:		**SECTION:**	
DATE PERFORMED:		**DATE SUBMITTED:**	
NAME:		**INSTRUCTOR:**	

ACTIVITY NO. ___ [ACTIVITY NAME]

1. PROCEDURE OUTPUT

What procedures did you perform?

ILO A: Create C++ code for graph implementation utilizing adjacency matrix and adjacency list

A.1. Create a Graph


A.2. Adjacency Matrix	
Code Snippet	Output

ILOA2.cpp > display_Matrix(DiaGraph &, int)

```cpp
#include <iostream>

// structure to store edges
struct graphEdge {
    int start_ver, end_ver, weight;
};

class DiaGraph {
    int N;  // number of vertices

public:
    int **matrix;  // adjacency matrix

    // Constructor
    DiaGraph(graphEdge edges[], int n, int N) {
        this->N = N;

        // create the matrix and set all values to 0
        matrix = new int *[N];
        for (int i = 0; i < N; i++) {
            matrix[i] = new int[N];
            for (int j = 0; j < N; j++)
                matrix[i][j] = 0;
        }

        // add the edges to the matrix
        for (int i = 0; i < n; i++) {
            int start_ver = edges[i].start_ver;
            int end_ver = edges[i].end_ver;
            int weight = edges[i].weight;

            matrix[start_ver][end_ver] = weight;
        }
    }

    // Destructor
    ~DiaGraph() {
        for (int i = 0; i < N; i++)
            delete[] matrix[i];
        delete[] matrix;
    }
};

// print the adjacency matrix
void display_Matrix(DiaGraph &graph, int N) {
    std::cout << "  ";
    for (int i = 0; i < N; i++)
        std::cout << char('A' + i) << " ";
    std::cout << std::endl;

    for (int i = 0; i < N; i++) {
        std::cout << char('A' + i) << " ";
        for (int j = 0; j < N; j++)
            std::cout << graph.matrix[i][j] << " ";
        std::cout << std::endl;
    }
}
```

```cpp
int main() {
    // graph edges array: {from, to, weight}  (A=0, B=1, C=2, D=3, E=4)
    graphEdge edges[] = {
        {0, 1, 2}, {0, 2, 4}, {1, 4, 3}, {2, 3, 2}, {3, 1, 4}, {4, 3, 3}
    };

    int N = 5;  // number of vertices
    int n = sizeof(edges) / sizeof(edges[0]);  // number of edges

    // create the graph
    DiaGraph diagraph(edges, n, N);

    // print the adjacency matrix
    std::cout << "Graph adjacency matrix:" << std::endl;
    display_Matrix(diagraph, N);

    return 0;
}
```

```
PS C:\Users\Administrator\Desktop\CPE21S1 - Villaruel\HOA 10>
Graph adjacency matrix:
  A B C D E
A 0 2 4 0 0
B 0 0 0 0 3
C 0 0 0 2 0
D 0 4 0 0 0
E 0 0 0 3 0
```

A.3. Adjacency List

Code Snippet	Output

ILOA3.cpp > DiaGraph > DiaGraph(graphEdge [], int, int)

```cpp
#include <iostream>
// Stores adjacency list items
struct adjNode {
    int val, cost;
    adjNode* next;
};

// Structure to store edges
struct graphEdge {
    int start_ver, end_ver, weight;
};

class DiaGraph {
    // Insert new nodes into adjacency list from given graph
    adjNode* getAdjListNode(int value, int weight, adjNode* head) {
        adjNode* newNode = new adjNode;
        newNode->val = value;
        newNode->cost = weight;
        newNode->next = head; // Point new node to current head

        return newNode;
    }

    int N; // Number of nodes in the graph

public:
    adjNode** head; // Adjacency list as array of pointers

    // Constructor
    DiaGraph(graphEdge edges[], int n, int N) {
        // Allocate new node
        head = new adjNode*[N]();
        this->N = N;

        // Initialize head pointer for all vertices
        for (int i = 0; i < N; ++i)
            head[i] = nullptr;

        // Construct directed graph by adding edges to it
        for (unsigned i = 0; i < n; i++) {
            int start_ver = edges[i].start_ver;
            int end_ver = edges[i].end_ver;
            int weight = edges[i].weight;

            // Insert in the beginning
            adjNode* newNode =
                getAdjListNode(end_ver, weight, head[start_ver]);

            // Point head pointer to new node
            head[start_ver] = newNode;
        }
    }
    // Destructor
    ~DiaGraph() {
        for (int i = 0; i < N; i++)
            delete[] head[i];

        delete[] head;
    }
};
// Print all adjacent vertices of given vertex
void display_AdjList(adjNode* ptr, int i) {
    while (ptr != nullptr) {
        std::cout << "(" << i << ", " << ptr->val
                  << ", " << ptr->cost << ") ";

        ptr = ptr->next;
    }

    std::cout << std::endl;
}
```

```
PS C:\Users\Administrator\Desktop\CPE21S1 - Villaruel\HOA 10>
Graph adjacency list
(start_vertex, end_vertex, weight):
(0, 2, 4) (0, 1, 2)
(1, 4, 3)
(2, 3, 2)
(3, 1, 4)
(4, 3, 3)
```

```cpp
// Graph implementation
int main() {
    // Graph edges array
    graphEdge edges[] = {
        // (x, y, w) -> edge from x to y with weight w
        {0, 1, 2},
        {0, 2, 4},
        {1, 4, 3},
        {2, 3, 2},
        {3, 1, 4},
        {4, 3, 3}
    };

    int N = 6; // Number of vertices in the graph

    // Calculate number of edges
    int n = sizeof(edges) / sizeof(edges[0]);

    // Construct graph
    DiaGraph diagraph(edges, n, N);

    // Print adjacency list representation of graph
    std::cout << "Graph adjacency list "
              << std::endl
              << "(start_vertex, end_vertex, weight):"
              << std::endl;

    for (int i = 0; i < N; i++) {
        // Display adjacent vertices of vertex i
        display_AdjList(diagraph.head[i], i);
    }

    return 0;
}
```

ILO B: Create C++ code for implementing graph traversal algorithms such as Breadth-First and Depth-First Search

B.1. Depth-First Search	
Code Snippet	Output

ILO81.cpp > Edge<T> > dest

```cpp
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <stack>

template <typename T>
class Graph;

template <typename T>
struct Edge
{
    size_t src;
    size_t dest;
    T weight;

    inline bool operator<(const Edge<T> &e) const
    {
        return this->weight < e.weight;
    }
    inline bool operator>(const Edge<T> &e) const
    {
        return this->weight > e.weight;
    }
};

template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
{

    for (size_t i = 1; i <= G.vertices(); i++)
    {
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto &e : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";
        os << std::endl;
    }
    return os;
}

template <typename T>
class Graph
{
public:
    Graph(size_t N) : V(N) {}

    auto vertices() const
    {
        return V;
    }

    const std::vector<Edge<T>> &edges() const
    {
        return edge_list;
    }

    void add_edge(Edge<T> &&e)
    {
        if (e.src >= 1 && e.src <= V &&
            e.dest >= 1 && e.dest <= V)
            edge_list.emplace_back(e);
        else
            std::cerr << "Vertex out of bounds" << std::endl;
    }

    auto outgoing_edges(size_t v) const
    {
        std::vector<Edge<T>> edges_from_v;
        for (auto &e : edge_list)
        {
            if (e.src == v)
                edges_from_v.emplace_back(e);
        }
        return edges_from_v;
    }

    template <typename U>
    friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);

private:
    size_t V;
    std::vector<Edge<T>> edge_list;
};
```

```
PS C:\Users\Administrator\Desktop\CPE21S1 - Villaruel\XOA 10>
1:      {2: 0}, {5: 0},
2:      {1: 0}, {5: 0}, {4: 0},
3:      {4: 0}, {7: 0},
4:      {2: 0}, {3: 0}, {5: 0}, {6: 0}, {8: 0},
5:      {1: 0}, {2: 0}, {4: 0}, {8: 0},
6:      {4: 0}, {7: 0}, {8: 0},
7:      {3: 0}, {6: 0},
8:      {4: 0}, {5: 0}, {6: 0},
9:

DFS Order of vertices:
1
5
8
6
7
3
4
2
```

```cpp
template <typename T>
auto depth_first_search(const Graph<T> &G)
{
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;

    stack.push(1); // DFS starting vertex

    while (!stack.empty())
    {
        auto current_vertex = stack.top();
        stack.pop();

        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);

            for (auto e : G.outgoing_edges(current_vertex))
            {
                if (visited.find(e.dest) == visited.end())
                {
                    stack.push(e.dest);
                }
            }
        }
    }
    return visit_order;
}

template <typename T>
auto create_reference_graph()
{
    Graph<T> G(9);
    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 0}, {5, 0}};
    edges[2] = {{1, 0}, {5, 0}, {4, 0}};
    edges[3] = {{4, 0}, {7, 0}};
    edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
    edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
    edges[6] = {{4, 0}, {7, 0}, {8, 0}};
    edges[7] = {{3, 0}, {6, 0}};
    edges[8] = {{4, 0}, {5, 0}, {6, 0}};

    for (auto &i : edges)
        for (auto &j : i.second)
            G.add_edge(Edge<T>(i.first, j.first, j.second));

    return G;
}

template <typename T>
void test_DFS()
{
    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;

    std::cout << "DFS Order of vertices: " << std::endl;
    auto dfs_visit_order = depth_first_search(G);
    for (auto v : dfs_visit_order)
        std::cout << v << std::endl;
}

int main()
{
    using T = unsigned;
    test_DFS<T>();
    return 0;
}
```

B.2. Breadth-First Search

Code Snippet	Output

ILO82.cpp

```cpp
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <queue>

using namespace std;

template <typename T>
class Graph;

template <typename T>
struct Edge
{
    size_t src;
    size_t dest;
    T weight;

    inline bool operator<(const Edge<T> &e) const
    {
        return this->weight < e.weight;
    }
    inline bool operator>(const Edge<T> &e) const
    {
        return this->weight > e.weight;
    }
};

template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
{
    for (auto i = 1; i < G.vertices(); i++)
    {
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto &e : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";
        os << std::endl;
    }
    return os;
}

template <typename T>
class Graph
{
public:
    Graph(size_t N) : V(N) {}

    auto vertices() const
    {
        return V;
    }

    auto &edges() const
    {
        return edge_list;
    }

    void add_edge(Edge<T> &&e)
    {
        if (e.src >= 1 && e.src <= V &&
            e.dest >= 1 && e.dest <= V)
            edge_list.emplace_back(e);
        else
            std::cerr << "Vertex out of bounds" << std::endl;
    }

    auto outgoing_edges(size_t v) const
    {
        std::vector<Edge<T>> edges_from_v;
        for (auto &e : edge_list)
        {
            if (e.src == v)
            {
                edges_from_v.emplace_back(e);
            }
        }
        return edges_from_v;
    }

    template <typename P>
    friend std::ostream &operator<<(std::ostream &os, const Graph<P> &G);

private:
    size_t V;
```

```
PS C:\Users\Administrator\Desktop\CPE21S1 - Villaruel\HOA 10>
1:      {2: 2}, {5: 3},
2:      {1: 2}, {5: 5}, {4: 1},
3:      {4: 2}, {7: 3},
4:      {2: 1}, {3: 2}, {5: 2}, {6: 4}, {8: 5},
5:      {1: 3}, {2: 5}, {4: 2}, {8: 3},
6:      {4: 4}, {7: 4}, {8: 1},
7:      {3: 3}, {6: 4},
8:      {4: 5}, {5: 3}, {6: 1},

BFS Order of vertices:
1
2
5
4
8
3
6
7
```

```cpp
    std::vector<Edge<T>> edge_list;
};

template <typename T>
auto create_reference_graph()
{
    Graph<T> G(9);
    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 2}, {5, 3}};
    edges[2] = {{1, 2}, {5, 5}, {4, 1}};
    edges[3] = {{4, 2}, {7, 3}};
    edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
    edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
    edges[6] = {{4, 4}, {7, 4}, {8, 1}};
    edges[7] = {{3, 3}, {6, 4}};
    edges[8] = {{4, 5}, {5, 3}, {6, 1}};

    for (auto &i : edges)
        for (auto &j : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});

    return G;
}

template <typename T>
auto breadth_first_search(const Graph<T> &G, size_t dest)
{
    std::queue<size_t> queue;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;

    queue.push(1); // Assume that BFS always starts from vertex ID 1

    while (!queue.empty())
    {
        auto current_vertex = queue.front();
        queue.pop();

        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);

            for (auto e : G.outgoing_edges(current_vertex))
            {
                if (visited.find(e.dest) == visited.end())
                {
                    queue.push(e.dest);
                }
            }
        }
    }
    return visit_order;
}

template <typename T>
void test_BFS()
{
    // Create an instance of and print the graph
    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;

    // Run BFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "BFS Order of vertices: " << std::endl;
    auto bfs_visit_order = breadth_first_search(G, 1);
    for (auto v : bfs_visit_order)
        std::cout << v << std::endl;
}

int main()
{
    using T = unsigned;
    test_BFS<T>();
    return 0;
}
```

COMPUTER ENGINEERING

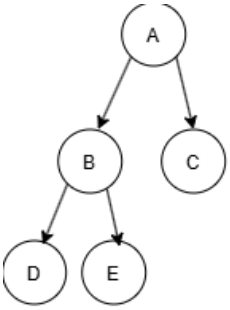
QUEZON CITY

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

ILO C: Demonstrate an understanding of graph implementation, operations and traversal methods.

Question	Answer
A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from same vertex. Discuss which algorithm would be most helpful to accomplish this task.	Depth-First Search (DFS) would be the most useful algorithm to use in order to accomplish this task. DFS explores a graph by going as deep as possible along each branch before backtracking to the previous node to check unexplored paths. This perfectly fits the given scenario when it comes to visiting every location until the path finishes, backing up, and trying another path.

Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.	
	``
	// DFS (preorder): Root, Left, Right
	void dfs(Node *n) {
	    if (n == nullptr) return;
	    std::cout << n->val << " "; // visit the node
	    dfs(n->left); // go deep on the left side
	    dfs(n->right); // then the right side
	}
	``
	Pseudocode
	Start
	if node is null, stop
	visit (node)
	PREORDER (left)
	PREORDER (right)
	End

In the performed code, what data structure is used to implement the Breadth First Search?	A queue, specifically std::queue<size_t>. It is in FIFO, so the code visits all neighbors of a vertex before moving one level deeper. BFS starts by pushing vertex 1, then repeatedly takes the vertex at the front and pushes its unvisited neighbor to the back.
How many times can a node be visited in the BFS?	A node can be visited only once. In the performed

	code, a vertex may be pushed into the queue more than once, for example vertex 4 can be queued from both vertex 2 and vertex 5. However, when it is taken out of the queue, the code checks the visited set.

Conclusion
In this hands-on activity, I learned how to represent a graph using an adjacency matrix and an adjacency list in C++, and how to traverse it using BFS and DFS. I created a weighted graph, and displayed it as a matrix and as a list. I also learned that DFS is best for exploring one path and backtracking, and that BFS uses a queue while tracking visited nodes. While I have a good grasp of how graph representations and traversals work conceptually, I still struggle with creating and implementing them in code. To improve, I plan to practice doing the setup and traversal algorithms again and again and try to watch online tutorials so that I can use it as a basis for future activities.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.